

Аргументы функций

В Python есть три распространенных типа аргумента функций:

- **Позиционные аргументы:** аргументы, которые вызываются по их положению в определении функции.
- **Именованные аргументы:** аргументы, вызываемые по имени.
- **Аргументы по умолчанию:** аргументы, которым присвоены значения по умолчанию.

Напомним, вот как выглядит каждый из этих типов аргументов:

- **Позиционные аргументы**

```
def print_name(first_name, last_name):  
    print(first_name, last_name)  
  
print_name('Даша', Пирогова')
```

Здесь `first_name` будет сопоставлено с «Jiho», а `last_name` будет сопоставлено с «Baggins» из-за положения аргументов при вызове функции.

- **Именованные аргументы**

```
def print_name(first_name, last_name):  
    print(first_name, last_name)  
  
print_name(last_name=' Пирогова ', first_name=' Даша ')
```

Здесь мы используем имена параметров `first_name` и `last_name` в качестве аргументов ключевого слова в вызове функции. Обратите внимание, что порядок аргументов не имеет значения, поскольку им присваивается определенное имя.

- **Аргументы по умолчанию**

```
def print_name(first_name='Jiho', last_name='Baggins'):  
    print(first_name, last_name)  
  
print_name()
```

Здесь, в определении функции, мы присваиваем параметрам значения по умолчанию. Это означает, что мы можем вызывать нашу функцию без каких-либо аргументов, потому что у них будет значение, к которому можно вернуться.

Хотя это наиболее распространенные типы аргументов, в этом уроке мы рассмотрим, что происходит, когда мы хотим сделать аргументы нашей функции более гибкими, принимая различное количество аргументов.

Прежде чем мы перейдем к более подробному изучению типов переменных аргументов, давайте рассмотрим наиболее распространенные типы, чтобы разогреться.

Хотя это наиболее распространенные типы аргументов, в этом уроке мы рассмотрим, что происходит, когда мы хотим сделать аргументы нашей функции более гибкими, принимая различное количество аргументов.

Задание

1. Наш друг Джихо пытается заняться ресторанным бизнесом. Он попросил нас создать простую программу назначения столиков, чтобы помочь управлять тем, какой клиент сидит за каждым столиком в ресторане. Джихо хочет сохранить не только имя посетителя стола, но и статус VIP (полученный за частое посещение ресторана).

Информация о столах будет храниться в словаре, называемом Tables (уже заданном в нашем редакторе), который структурирован в следующем формате:

Table number: [имя, vip_status]

```
tables = {  
    1: ['Jiho', False],  
    2: [],  
    3: [],  
    4: [],  
    5: [],  
    6: [],  
    7: [],  
}
```

2. Джихо нужен способ назначать новых клиентов за существующие столики, когда они приходят зарезервировать столик. Определите новую функцию с именем `assign_table`, которая будет принимать три аргумента (именно в таком порядке):

- `table_number`
- `name`
- `vip_status`

Затем наша функция `assign_table` должна использовать данные аргументы для назначения нового клиента столу в созданном словаре. Используйте `table_number` в качестве ключа и список, содержащий `name` и `vip_status` в качестве значения.

3. Похоже, наша первая бронь только что поступила!

Вызовите нашу функцию `assign_table`, используя позиционные аргументы для клиента с именем «Yoni» за стол 6. Yoni не имеет VIP-статуса и, следовательно, должно иметь значение `False`.

4. Поскольку большинство клиентов в ресторане впервые, мы хотим использовать значение `False` по умолчанию для любого нового назначения стола.

Измените определение функции `assign_table` и присвойте параметру `vip_status` значение по умолчанию `False`.

5. Только что пришел новый клиент. Добавьте клиента в таблицу, используя следующую информацию через позиционные аргументы:

- Имя: «Карла»
- Номер стола: 4

Карла не является VIP, поэтому нам не нужно предоставлять дополнительный аргумент, поскольку в нашем определении функции уже есть значение по умолчанию `False`.

6. Наконец, выведите словарь, чтобы увидеть окончательный вариант рассадки посетителей.

Переменное количество аргументов: *args

Чтобы начать изучение динамического (изменяемого) количества аргументов в функциях Python, давайте взглянем на знакомую функцию, которую мы использовали в течение длительного времени:

```
print('This', 'is', 'the', 'print', 'function')
```

Обратите внимание, что функции print() все равно, сколько аргументов мы ей передаем. Итак, вопрос в том, как print() может принимать на вход любое количество переменных?

Что ж, в Python есть дополнительный оператор, называемый оператором распаковки (парсинга) (*). Оператор распаковки позволяет нам давать нашим функциям переменное количество аргументов, выполняя так называемую позиционную упаковку аргументов.

Давайте рассмотрим, как это работает, изучив базовую функцию, использующую оператор распаковки:

```
def my_function(*args):  
    print(args)
```

Если бы мы вызвали эту функцию со случайными аргументами:

```
my_function('Arg1', 245, False)
```

Результат выведет, что находится внутри *args:

```
('Arg1', 245, False)
```

Обратите внимание на две вещи:

- В нашем вызове print() мы использовали имя args с оператором распаковки. Имя args было дано произвольно, и этот пример работает точно так же с любым другим именем:

```
def my_function(*randomname):  
    print(randomname)
```

- Независимо от того, какое имя следует за оператором распаковки (*), аргументы, переданные в функцию, будут сохранены в форме кортежа. Это позволяет нашим функциям принимать любое количество аргументов, как и функция print(), которую мы рассмотрели ранее. В этом случае args имеет внутри три значения, но их может быть гораздо больше (или меньше).

Задание

1. Джихо хочет расширить наше ресторанное приложение, чтобы оно также принимало заказы от клиентов. Это возможно реализовать с использованием оператора распаковки, поскольку мы никогда не знаем, сколько товаров собираются заказать покупатели.

Для начала мы хотим создать функцию, которая будет составлять список всех товаров, которые клиент хочет заказать, а затем выводить его в консоль. Это поможет нашей кухне узнавать, что нужно готовить в рамках заказов посетителей.

Определите функцию с именем `print_order()`, которая будет принимать переменное количество аргументов, используя параметр с именем `order_items`. Функция должна просто выводить `order_items`.

2. Кажется, пришел наш первый заказ! Вызовите нашу функцию `print_order()` со следующими элементами заказа:

```
'Orange Juice'  
'Apple Juice'  
'Scrambled Eggs'  
'Pancakes'
```

Работа с *аргументами

Теперь, когда мы рассмотрели основы работы с распаковкой позиционных аргументов, давайте рассмотрим, как использовать ее более осмысленно.

Скажем, мы хотели создать функцию, которая работает аналогично оператору `print()`, но вместо этого выводит все аргументы в верхнем регистре. Используя наши знания о циклах в сочетании с мощностью оператора распаковки, наше решение может выглядеть так:

```
def shout_strings(*args):  
    for argument in args:  
        print(argument.upper())  
  
shout_strings('Working on', 'learning', 'argument unpacking!')
```

Результат исполнения:

```
WORKING ON  
LEARNING  
ARGUMENT UNPACKING!
```

В нашей функции `Shout_strings()` мы используем цикл `for` (хотя мы могли бы использовать любой итератор) для перебора каждого элемента кортежа, который существует внутри `args`. Затем мы используем встроенную функцию `.upper()`, чтобы использовать заглавные буквы для каждого аргумента.

Оператор распаковки можно использовать не только отдельно, но и в сочетании с другими позиционными аргументами. Давайте рассмотрим функцию, которая обрезает предложения на определенной длине:

```
def truncate_sentences(length, *sentences):  
    for sentence in sentences:  
        print(sentence[:length])  
  
truncate_sentences(8, "What's going on here", "Looks like we've been  
cut off")
```

Выдаст:

```
What's g  
Looks li
```

Что делает этот код:

- У нас есть два параметра, которые определяет наша функция `truncate_sentences()`. Первый — это параметр длины, который указывает, сколько символов мы хотим сохранить. Второй — это параметр, называемый `sentences`, который связан с оператором распаковки, что означает, что он будет принимать переменное количество аргументов.

- На каждой итерации функции мы перебираем кортеж, созданный аргументом `sentences` (поскольку он связан с оператором распаковки), и выполняем срез предложения на основе предоставленного аргумента длины. Это приводит к сокращению длины каждой строки в кортеже `sentences`.

Использование итерации и других позиционных аргументов — это два распространенных способа повысить полезность функций при использовании оператора распаковки (*).

Задание

1. Jiho имеет большой успех с нашим ресторанным приложением. К сожалению, наш первоначальный дизайн не учитывал хранение заказов для каждого конкретного стола. Джихо попросил нас настроить наше приложение, чтобы иметь возможность хранить заказы, поступающие для каждого конкретного стола, а также иметь возможность распечатывать заказы для кухонного персонала.

просмотрите скорректированную структуру программы, которую мы создали ранее. Обратите внимание, что `tables` теперь являются словарем словарей с номерами столов в качестве ключей. Он также учитывает новое свойство, называемое заказом. Функция `assign_table` также была скорректирована с учетом изменений.

```
tables = {
    1: {
        'name': 'Jiho',
        'vip_status': False,
        'order': 'Orange Juice, Apple Juice'
    },
    2: {},
    3: {},
    4: {},
    5: {},
    6: {},
    7: {},
}
print(tables)
```

2. Чтобы помочь Jiho реализовать возможность сохранения заказа в определенной таблице, давайте реализуем функцию с именем `assign_and_print_order()`.

Функция должна принимать два параметра с именами `table_number` и `order_items`. Параметр `order_items` должен быть сгруппирован с оператором распаковки (*), чтобы мы могли зафиксировать переменное количество позиций заказа на таблицу.

3. Затем функция `assign_and_print_order()` должна внести заказ словарь соответствующему столу. Внутри нашей функции получите доступ к вложенному ключу `order` для конкретного стола (используя аргумент `table_number`) из таблиц и установите его в параметр `order_items`.
4. Затем мы также хотим вывести каждый заказанный элемент, чтобы кухня знала, что готовить!

Внутри `assign_and_print_order()` используйте цикл `for` для перебора `order_items` и вывода каждого отдельного элемента заказа.

5. Наконец, давайте посмотрим на нашу функцию в действии. К счастью, у нас только что появился новый клиент, который забронировал стол. Используйте функцию `assign_table()`, чтобы добавить нового клиента с именем «Arwa» в таблицу 2 с VIP-статусом, установленным на `True`.
6. Теперь, когда «Arwa» готов сделать заказ, вызовите нашу функцию `assign_and_print_order()` для стола 2 с элементами заказа «Стейк», «Морской окунь» и «Бутылка вина».

Динамическое количество аргументов: ****kwargs**

Python не ограничивается тем, что позволяет нам принимать неограниченное количество позиционных аргументов; это также дает нам возможность определять функции с неограниченным количеством именованных аргументов. Синтаксис очень похож, но использует две звездочки `**` вместо одной. Обычно мы называем эти `kwargs` сокращением именованные аргументы (Key arguments).

Давайте рассмотрим функцию, которая выводит некоторую полезную информацию о `kwargs`, чтобы увидеть ее в действии:

```
def arbitrary_keyword_args(**kwargs):
    print(type(kwargs))
    print(kwargs)
    # See if there's an 'anything_goes' keyword arg and print it
    print(kwargs.get('anything_goes'))
```

```
arbitrary_keyword_args(this_arg='wowzers', anything_goes=101)
```

Данный код выведет:

```
<class 'dict'>
{'this_arg': 'wowzers', 'anything_goes': 101}
101
```

Можно заметить две вещи:

- `**kwargs` принимает форму словаря, в котором все значения аргументов ключевого слова передаются в `random_keyword_args`. Поскольку `**kwargs` — это словарь, мы можем использовать стандартные функции словаря, такие как `.get()`, для извлечения значений.
- Так же, как мы видели с `*args`, имя `kwargs` совершенно произвольно, и этот пример работает точно так же с любым другим названием аргумента:

```
def arbitrary_keyword_args(**data):
    # ...
```

Задание

1. Джихо доволен тем, как мы можем хранить заказы для наших столов. Однако персонал теперь хочет различать продукты питания и напитки.

Поскольку продукты готовятся на кухне, а напитки готовятся в баре, в нашем приложении важно различать их.

Словарь столов был изменен для поддержки запросов персонала.

```

tables = {
    1: {
        'name': 'Chioma',
        'vip_status': False,
        'order': {
            'drinks': 'Orange Juice, Apple Juice',
            'food_items': 'Pancakes'
        }
    },
    2: {},
    3: {},
    4: {},
    5: {},
    6: {},
    7: {},
}

```

2. Поскольку наша программа теперь требует различать продукты питания и напитки, это отличное место для использования возможностей `**kwargs`.

Определите функцию `assign_food_items()` с одним параметром, `order_items`. Соедините этот параметр с оператором `**` для обработки любых именованных аргументов.

3. Теперь мы хотим записать продукты и напитки в `order_items`. Используйте метод `.get()`, чтобы сделать следующее:

- Назначьте значения именованного аргумента `food` переменной `food`.
- Назначьте значения именованного аргумента `drink`, и назначьте их переменной, называемой `drinks`.

Вызовите функцию

```
assign_food_items(food='Pancakes, Poached Egg', drinks='Water')
```

Работа с `**kwargs`

Работа с `**kwargs` очень похожа на работу с `*args`. Поскольку `**` генерирует стандартный словарь, мы можем использовать итерацию так же, как и раньше, воспользовавшись методом `.values()`. Вот пример:

```

def print_data(**data):
    for arg in data.values():
        print(arg)

print_data(a='arg1', b=True, c=100)

```

выведет:

```
arg1
True
100
```

Мы также можем комбинировать использование `**` с обычными позиционными аргументами. Однако Python требует, чтобы все позиционные аргументы стояли первыми в определении нашей функции. Давайте посмотрим, как это работает:

```
def print_data(positional_arg, **data):
    print(positional_arg)
    for arg in data.values():
        print(arg)

print_data('position 1', a='arg1', b=True, c=100)
```

Выведет:

```
position 1
arg1
True
100
```

Если бы мы поменяли позицию `positional_arg` так, чтобы она шла после `**data`, мы бы столкнулись с `SyntaxError`.

Давайте расширим наше ресторанное приложение из предыдущих упражнений, чтобы применить гибкость использования `**kwargs` в наших функциях.

Объединение всех видов аргументов

До сих пор мы видели, как `*args` и `**kwargs` могут сочетаться со стандартными аргументами. Это полезно, но в некоторых случаях мы можем захотеть использовать все три типа вместе! К счастью, Python позволяет нам это делать, если мы следуем правильному порядку в определении нашей функции. Порядок, следующий:

1. Стандартные позиционные аргументы
2. `*args`
3. Стандартные аргументы ключевых слов
4. `**kwargs`

Например, вот как могло бы выглядеть определение нашей функции, если бы нам нужна была функция, печатающая животных, используя все три типа:

```
def print_animals(animal1, animal2, *args, animal4, **kwargs):
    print(animal1, animal2)
    print(args)
    print(animal4)
    print(kwargs)
```

Мы могли бы вызвать нашу функцию так:

```
print_animals('Snake', 'Fish', 'Guinea Pig', 'Owl', animal4='Cat',
animal5='Dog')
```

```
Snake Fish
('Guinea Pig', 'Owl')
Cat
{'animal5': 'Dog'}
```

Давайте разберем:

- Первые два аргумента, которые принимает наша функция, будут принимать форму стандартных позиционных аргументов. Когда мы вызываем функцию, первые два предоставленных значения сопоставляются с `animal1`, `animal2`. Таким образом, первая строка вывода — Snake Fish.
- Аргументы, не являющиеся именованными, которые следуют после Snake и Fish в вызове нашей функции, отображаются в кортеж `args`. Таким образом, наш результат ('Морская свинка', 'Сова')
- Затем мы переходим к обычным именованным аргументам. Поскольку мы назвали `animal4` в качестве именованного аргумента, наш результат для оператора печати — Кошка.
- Наконец, у нас есть еще один аргумент ключевого слова, который сопоставляется с `**kwargs`. Таким образом, наша последняя строка вывода — {'animal_5': 'Dog'}

Распаковка вызовов функций и не только

Надеюсь, к настоящему моменту мы начали видеть возможности использования операторов `*` и `**` в наших определениях функций. Однако Python не останавливается на достигнутом! Мы можем использовать операторы не только при определении параметров, но и в вызовах функций.

Давайте представим, что мы хотим суммировать несколько чисел вместе:

```
def sum(num1, num2, num3):
    print(num1 + num2 + num3)
```

Прямо сейчас наша функция заставляет нас передавать отдельные аргументы для `num1`, `num2` и `num3`. Это не большая проблема, если у нас есть отдельные переменные или мы заранее знаем наши числа. Однако, что, если вместо этого мы захотим использовать список типа `[3, 6, 9]`? И тут на помощь приходит оператор распаковки.

Давайте посмотрим:

```
my_num_list = [3, 6, 9]

def sum(num1, num2, num3):
    print(num1 + num2 + num3)

sum(*my_num_list)
```

Выведет:

Итак, что здесь происходит? Что ж, с помощью оператора распаковки (*) мы распределяем содержимое нашего списка `my_num_list` по отдельным аргументам в определении нашей функции. Мы сразу же избавлены от необходимости писать циклы и можем использовать любой итерируемый объект с тремя элементами.

Этот способ использования * в вызове функции также применим к нашему ключевому оператору **. Пока ключевые слова совпадают с именами параметров функции, мы можем достичь той же цели:

```
numbers = {'num1': 3, 'num2': 6, 'num3': 9}
```

```
def sum(num1, num2, num3):  
    print(num1 + num2 + num3)
```

```
sum(**numbers)
```

Мы даже можем использовать операторы внутри встроенных функций. Например, вместо того, чтобы вручную указывать для встроенной функции `range()` начальное и конечное значения, мы можем распаковать список прямо в нее. Давайте взглянем:

```
start_and_stop = [3, 6]
```

```
range_values = range(*start_and_stop)  
print(list(range_values))
```

Выведет

```
[3, 4, 5]
```

Возможности использования операторов * и ** безграничны. Обратите внимание на еще несколько умных вариантов использования:

- Распаковка частей итерируемого

```
a, *b, c = [3, 6, 9, 12, 15]  
print(b)
```

- Объединение итераций

```
my_tuple = (3, 6, 9)  
merged_tuple = (0, *my_tuple, 12)  
print(merged_tuple)
```

- Сочетание распаковки и упаковки

```
num_collection = [3, 6, 9]
```

```
def power_two(*nums):  
    for num in nums:  
        print(num**2)
```

```
power_two(*num_collection)
```

Выведет:

```
9  
36  
81
```

Задание

1. Джихо считает, что нашему ресторанному приложению не хватает одной очень важной функции. Джихо хотел бы, чтобы приложение могло рассчитать общий счет за стол (включая чаевые) и разделить его в зависимости от количества людей за столом.

К счастью, у нас была существующая функция `calculate_price_per_person()` из нашего последнего ресторанного проекта, которую мы можем использовать повторно (смотри ниже). Потратьте некоторое время, чтобы изучить функцию и ее внутреннюю работу.

```
def calculate_price_per_person(total, tip, split):  
    total_tip = total * (tip/100)  
    split_price = (total + total_tip) / split  
    print(split_price)
```

2. Похоже, мы готовы протестировать нашу функцию! К счастью, седьмой столик в ресторане Джихо готов к оплате.

Определите список с именем `table_7_total`, который имеет следующие значения по порядку:

1. 534,50 (Общая стоимость счета)
 2. 20,0 (процент чаевых)
 3. 5 (количество человек, на которых нужно разделить счет)
3. Используя оператор распаковки, вызовите `calculate_price_per_person()` со списком `table_7_total`.